

Réponses du test

Utilisation de la solution (étape 1 à 3)

Inscrire la documentation technique

Étape 1 : Création de l'environnement virtuel

Pour la création de l'environnement virtuel, nous avons fait différentes commandes pour faire le tout.

Nous voulons nous assurer que venv est installé pour pouvoir exécuter la prochaine commande:

```
python -m venv --help
```

Une fois que la commande retourne une aide sur venv, nous pouvons créer notre environnement virtuel avec cette commande:

```
python -m venv venv
```

Cette commande créera un nouveau dossier venv avec plusieurs documents différents. Une fois le document créé, on exécute cette commande.

```
venv\Scripts\activate
```

Il est possible que par défaut, il y a une politique d'exécution des scripts sous PowerShell, alors Windows bloque l'exécution des fichiers .ps1. Il faut donc autoriser temporairement l'exécution des scripts

```
Set-ExecutionPolicy Unrestricted -Scope Process
```

Une fois l'environnement virtuelle activé, on peut installer les différents dépendances nécessaires dans le fichier requirements.txt en faisant la commande:

```
pip install -r requirements.txt
```

Étape 2:

Nous utilisons `schtasks` pour exécuter le script chaque jour à 08h00 du matin :

```
schtasks /create /tn "FetchAPIData" /tr "\"C:\\Users\\Clayton\\git\\technical-
test-data-engineer\\venv\\Scripts\\python.exe\"
\"C:\\Users\\Clayton\\git\\technical-test-data-
engineer\\src\\moovitamix_fastapi\\fetch_and_save.py\"" /sc daily /st 08:00 /F
```

Nous pouvons s'assurer que la tâche a bien été créée avec cette commande:

```
schtasks /query /tn "FetchAPIData"
```

Si tout fonctionne correctement, les fichiers CSV seront mis à jour dans le dossier datas. Il faut un serveur toujours actif.

Étape 3:

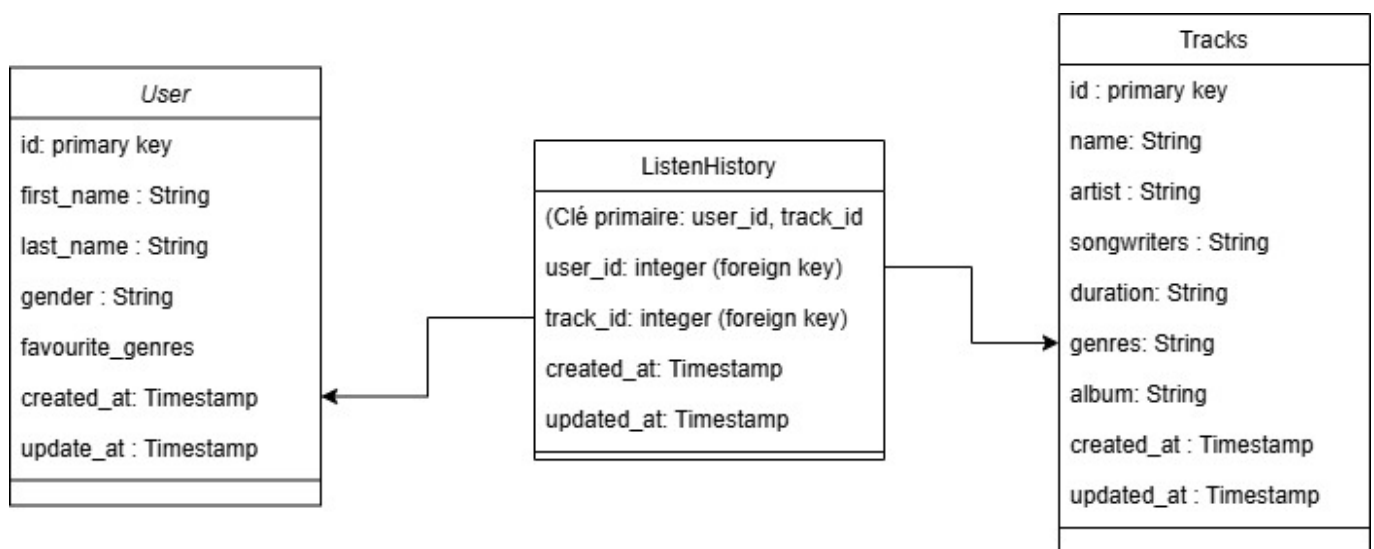
Pour s'assurer que le pipeline de récupération de données fonctionne correctement, nous avons ajouté des tests unitaires avec `pytest`

-

Ensuite nous avons ajout la configuration pour avoir un pre-commit. Ainsi, à chaque commit, pytest sera automatiquement exécuté pour tous les tests unitaires

Questions (étapes 4 à 7)

Étape 4



Voici mon schéma de base de données que j'utiliserais pour stocker les informations récupérés des trois sources données. La table **Users** possède les différents attributs ainsi qu'une clé primaire qui est le id de chacun des users. Par la suite, la table **Tracks** possède les différents attributs de la chanson ainsi qu'une clé primaire qui est le id. Pour la table **ListenHistory**, elle fait la relation entre **Users** et **Tracks**. La table joue le rôle d'être une table de jointure entre les 2 tables qui est une relation "many-to-many", ce qui veut dire que un utilisateur écoute plusieurs chansons et qu'une chanson est écoutée par plusieurs utilisateurs. Ainsi,

comme affiché dans le schéma, on aurait un clé étrangère venant de la table **Users** ainsi qu'une clé étrangère de la Table **Tracks**. La clé primaire sera l'unicité des deux clés étrangères ensemble.

Je recommanderais comme système de base de données PostgreSQL pour ainsi avoir un modèle relationnel qui correspond au schéma. PostgreSQL gère efficacement les jointures complexes et les relations "many-to-many", essentielles pour récupérer les morceaux écoutés par un utilisateur, l'historique, ou les statistiques d'écoute. Aussi, PostgreSQL permet de stocker des listes, JSON et tableaux, utiles si jamais on veut stocker des métadonnées flexibles. Considérant les prochaines étapes où nous voulons avoir un système de recommandation, cette base de données est la meilleure option. De plus, PostgreSQL fonctionne bien avec Python (Pandas, Scikit-learn) et SQL avancé pour des recommandations basées sur l'historique d'écoute.

À long terme, si nous envisageons avoir des gros volumes de donnée, PostgreSQL possède de l'indexation avancé pour une meilleur performance et le partage de charge avec des indexes et partitions pour accélérer les requêtes sur de gros volumes de données.

Étape 5

Pour assurer un suivi efficace du pipeline de données, nous utilisons **AWS CloudWatch** afin de collecter, analyser et visualiser les métriques essentielles en temps réel. CloudWatch permet de détecter rapidement les anomalies, d'envoyer des alertes automatiques en cas d'échec et d'optimiser la performance du pipeline.

En complément, une table spécifique pipeline_monitoring est intégrée dans la base de données pour stocker l'historique des exécutions. Cette table enregistre les métriques clés suivantes :

- Disponibilité – Taux de succès (%) : Mesure le pourcentage de jobs exécutés avec succès.
- Fiabilité – Taux d'échec (%) : Indique le pourcentage de jobs ayant échoué, permettant d'identifier les pannes récurrentes.
- Performance – Temps d'exécution (s) : Suivi du temps moyen d'exécution des jobs pour détecter les ralentissements.
- Volume – Nombre de lignes traitées : Permet de repérer des anomalies en identifiant des variations inattendues du volume de données ingérées.

Étape 6

votre réponse ici

Étape 7

votre réponse ici